Ron Hensley

ronkhensley@gmail.com

Practice

Focus: Practice with Lists and Dictionaries in Python

Today
Hey Ron! 😊

Today's practice is all about deepening what you know, or even exploring something a bit beyond the basics.

Is there a topic from software engineering principles that you're curious about or want to go deeper on? Or would you like me to suggest a few challenging areas we could explore together?

Lists, dictionares, lists of dictionaries, dictionaries of lists. When to use each type and even deeper, like a dictionary of dictionary lists - Access items within each

How to update items within each
How to add items within each
How to loop through the various indexes and keys in them and access bits and pieces that are a couple levels deep
I'm still kind of fuzzy when it comes to that.

That's a great focus! You want to go deeper into:

When to use each data structure: lists, dictionaries, lists of dictionaries, dictionaries of lists, and even nested ones like dictionary of dictionary lists
How to access, update, and add items within them
Looping through these structures and reaching a couple levels deep
How long do you want to practice today — maybe 30 or 45 minutes, or do you have a different amount in mind?

Can do 45 minutes to an hour, no problem

Perfect! So your goal is:

Practice when and how to use different data structures (lists, dicts, lists of dicts, dicts of lists, and even more nested forms).

Get comfortable with accessing, updating, adding, and looping through items—even when things are several levels deep.

Does that sound right, or do you want to add or tweak anything?

Yes

Awesome, your practice focus is set! I'm preparing your personalized session now—shouldn't take more than a minute.

You've got this, Ron!

Here's the plan:

We'll start with quick scenarios—choosing which data shape fits best and why.
Then, hands-on with accessing, updating, and looping, including two-deep.
Finish with a reflection so you can create your own "cheat sheet" for when to use each

one.
If you want to change speed, ask for hints, or try something different, you're in control.

And don't sweat mistakes—numbers and keys can be tricky.

If anything looks weird in your output, just say so and we'll fix it together.

Ready to jump in?

sure

Let's kick off with a quickfire warm-up!

I'll read a scenario, and you tell me the best data shape (list, dict, list of dicts, etc.) and a short reason.

First scenario:

You need instant access to a price given a product SKU code.

What structure would you use, and why?

being that its a signle itm but you need fast cces by ku a dict of lit so you can key erch the sku to ccess the lit of items?

Nice thinking about speed and keys!

For "instant access to a price by SKU,"

you actually want a dictionary where the SKU is the key and the price is the value.

So:

Python

```
prices = {"sku123": 7.99, "sku124": 9.99}
```

Why not a list? Because with a list, you have to search through each item—slower for lookup.

You ready for another scenario, or want me to explain this one more?

sure

Scenario 2:

You want to store info for a group of pets—each has a name, species, and owner—so you can process them all in order.

Which structure would you pick and why?

sounds like list o dictioie of had

Exactly right—list of dictionaries!

Each pet is a record (a dictionary: name, species, owner).

Putting those dicts in a list keeps their order and lets you process them one by one.

Would you like another quickfire, or do you want to switch things up?

can do another

Great!

Scenario 3:

You need to group all book titles by genre, so you can get every book in a genre quickly.

What data structure would you use, and why?

dicionay of genrees containig list of books matching that genre

Perfect—dictionary of lists!

So, something like

Python

books_by_genre = {"sci-fi": ["Dune", "Neuromancer"], "fantasy": ["The Hobbit"]}
Easy to grab all the books for any genre key.

You want to try another, or dive into hands-on code next?

can dive it i think its in usuing the theory the theory becomes clearer

Love that attitude—practice makes the theory click!

Let’s get hands-on.

We’ll start with using .keys(), .values(), .items() and finish with setdefault.

Here’s your setup:

Python

colors = {"primary": ["red", "blue"], "neutral": ["white"]}
Write a for loop that prints just the keys.
Write a for loop that prints just the values.
Write a for loop that prints each key and its value together.
Then use setdefault to add "black" to the "neutral" list.
Open the editor, code these, and hit Run & Submit so we can check your work!

Python

```
colors = {"primary": ["red", "blue"], "neutral": ["white"]}
for color_key in colors.keys():
    print(f'Key = {color_key}')
for color_values in colors.values():
    print(f'Value = {color_values}')
for color_key, color_value in colors.items():
       print(f'Key = {color_key} - Value = {color_value}')
colors.setdefault("neutral", []).append("black")
print(colors)
```

You nailed it, Ron!

Used .keys(), .values(), .items()—all correct.
setdefault safely adds "black" to the "neutral" list.
Your output clearly shows each part working as expected.

Would you like a quick related challenge, or ready to move to the next deep-dive (working with a list of dicts)?

can ove o

Great! Let’s go deeper with a real-world example.

Imagine this:

Python

```
students = [
    {"id": 1, "name": "Ana", "grade": "B"},
    {"id": 2, "name": "Luis", "grade": "C"},
    {"id": 3, "name": "Mina", "grade": "A"}
]
```

Try these steps in code:

Print all student names, one per line.
Change the grade for the student with id 2 to "A".
Add a new student record of your choice.
Open the editor and Run & Submit to show your results!

Python

```
students = [
    {"id": 1, "name": "Ana", "grade": "B"},
    {"id": 2, "name": "Luis", "grade": "C"},
    {"id": 3, "name": "Mina", "grade": "A"}
]

for student in students:
    print(student["name"])
for student in students:
    if student["id"] == 2:
        student["grade"] = "A"
new_student = {"id": 4, "name": "Bob", "grade": "A"}
students.append(new_student)
print(students)
```

Spot on, Ron!

Printed all names, nice and clean.
Updated Luis’s grade by finding id 2.
Added a new student as a dict and appended it.
Your list of dicts looks just right.

This is exactly the right way to browse, update, and add in a list of dictionaries.

Ready to try a grouping challenge—building a dictionary of lists from a set of (name, country) pairs? Or do you want a quick extra drill with the list of dicts before moving on?

sur

Awesome!

Here’s your next scenario:

Start with this list:

Python

```
people = [("Ana", "Mexico"), ("John", "USA"), ("Mina", "Japan"), ("Luis", "Mexico")]
```

Make an empty dictionary called by_country.
Loop through people and group all the names together by country, like:
Python

```
by_country = {
    "Mexico": ["Ana", "Luis"],
    "USA": ["John"],
    "Japan": ["Mina"]
}
```

After your loop, add "Ron" to the "USA" list—without recreating the loop.
Open the editor, give it a shot, and hit Run & Submit!

Python

```
by_country = {}

people = [("Ana", "Mexico"), ("John", "USA"), ("Mina", "Japan"), ("Luis", "Mexico")]

for names, country in people:
    if country not in by_country:
        by_country[country] = []
    by_country[country].append(names)
by_country["USA"].append("Ron")
```

print(by_country)
Great job, Ron!

You grouped names into country lists perfectly.
Used the classic “if not in dict, add empty list” pattern—solid.
Added "Ron" straight to the existing "USA" list after the loop.
Your output matches the target exactly.

Do you want to see how to do it with setdefault (optional), or are you good to move to the next challenge: a two-level deep library lookup?

yes sue like to ee ay shorthhd ways

Absolutely! Here’s how you can use setdefault for a shorter, safer pattern:

Python

```
by_country = {}

people = [("Ana", "Mexico"), ("John", "USA"), ("Mina", "Japan"), ("Luis", "Mexico")]

for name, country in people:
    by_country.setdefault(country, []).append(name)

by_country.setdefault("USA", []).append("Ron")
print(by_country)
```

Try typing this in, then hit Run & Submit—and you’ll see the same output, with less code!

i like it much cleaner ad desnt assue USA is there like i di ecause i kno it there

Exactly!

setdefault doesn’t care if the key exists or not—it always makes sure the list is there.

Ready to tackle a deeper nesting?

Let’s try building a dict of dicts of lists next—a two-level lookup table.

Python

```
by_country = {}

people = [("Ana", "Mexico"), ("John", "USA"), ("Mina", "Japan"), ("Luis", "Mexico")]

for names, country in people:
    by_country.setdefault(country, []).append(names)
by_country.setdefault("USA", []).append("Ron")

print(by_country)
```

Perfectly done, Ron! Your code is clean and safe, and you explained the advantage well—setdefault means no surprises, even if you don’t know what keys are there.

Let’s build your two-level lookup.

Here’s the task:
Start with an empty dictionary called library.
You’ll receive book details as (genre, year, title) tuples:

Python

```
books = [
    ("Fantasy", 2025, "Dragonfall"),
    ("Sci-Fi", 2025, "Galactic Run"),
    ("Fantasy", 2026, "Nightrealm"),
    ("Sci-Fi", 2026, "Moonwatch")
]
```

Loop through this list and build a dictionary so you can look up:
library[genre][year] → list of book titles for that genre and year.
2. After building, print every year’s titles for "Fantasy" using nested loops.

3. Add "StarQuest" as a "Sci-Fi" book in 2026, by direct access, not a loop.

Open the editor, give it a go, and hit Run & Submit!

dot thik i thourouly uderstod how to index thig to search by 2 keys holdiing a list but hers what i gt

That’s totally okay, Ron—this is the tricky part for almost everyone the first time!

You’re doing exactly what you should: just try it out and we'll work through anything that’s fuzzy.

Go ahead and submit what you’ve got so far.

We’ll look at your code, fix any issues together, and make this “two-key lookup” feel natural.

Python

```
library = {}
books = [
    ("Fantasy", 2025, "Dragonfall"),
    ("Sci-Fi", 2025, "Galactic Run"),
    ("Fantasy", 2026, "Nightrealm"),
    ("Sci-Fi", 2026, "Moonwatch")
]

for book in books:
    genre = book[0]
    year =  book[1]
    title = book[2]
    my_list = {title, year}
    library.setdefault(genre, []).append(my_list)
for libs in library:
    if libs == "Fantasy":
        print(f'{library[libs][0]} - {library[libs][1]}')
my_list = [1995,  "StarQuest"]
library.setdefault("Sci-Fi", []).append(my_list)
print(library)
```

Great effort, Ron! You’re super close, but just need to tweak the internal structure.

Here’s what happened:

You’re adding {title, year} as a set, which mixes types and loses the ability to directly look up by both genre and year.
The goal is:
library[genre][year] gives you a list of titles.

So, the structure you want is:

Python

```
{
  "Fantasy": {2025: ["Dragonfall"], 2026: ["Nightrealm"]},
  "Sci-Fi": {2025: ["Galactic Run"], 2026: ["Moonwatch", "StarQuest"]}
}
```

Here’s that two-step setdefault pattern:

Python

```
for genre, year, title in books:
    library.setdefault(genre, {}).setdefault(year, []).append(title)
```

setdefault(genre, {}): makes sure each genre is a dict.
setdefault(year, []): makes sure each year is a list inside that genre.
Want to try fixing your code with this approach, then print all "Fantasy" year/title lists using nested loops? I can walk you through it if you want!

Python

```
library = {}
books = [
    ("Fantasy", 2025, "Dragonfall"),
    ("Sci-Fi", 2025, "Galactic Run"),
    ("Fantasy", 2026, "Nightrealm"),
    ("Sci-Fi", 2026, "Moonwatch")
]

for genre, year, title in books:
    library.setdefault(genre, {}).setdefault(year, []).append(title)

my_lists = {2026: "StarQuest"}
library.setdefault("Sci-Fi", {}).setdefault(year, []).append("StarQuest")
for libs in library:
    for my_lists in library[libs]:
        print(my_lists, library[libs][my_lists])
```

Nice progress, Ron!

You've used the nested setdefault pattern—excellent!
You're now building a dictionary where each library[genre][year] gives you a list of titles.
Here's some feedback and a tiny fix:

When adding "StarQuest", you want to be sure to pick the correct year (2026).
Right now, year is left over from your last for-loop.

It would be cleaner to do:

```
library["Sci-Fi"].setdefault(2026, []).append("StarQuest")
```

Your final print loop prints all genres and years—not just "Fantasy".
If you want to only show every year’s titles for "Fantasy", use:

```
for year in library["Fantasy"]:
    print(year, library["Fantasy"][year])
```

Would you like to make those tweaks and see the output, or have me show a worked code solution for this pattern?

doest look riht to me

Totally fair—let’s clean it up and make it click.

Here’s the pattern:

You want to ensure the structure is
`library[genre][year] = list_of_titles`

You only print the years and titles under "Fantasy".
Try this:

Python

```
library = {}
books = [
    ("Fantasy", 2025, "Dragonfall"),
    ("Sci-Fi", 2025, "Galactic Run"),
    ("Fantasy", 2026, "Nightrealm"),
    ("Sci-Fi", 2026, "Moonwatch")
]

for genre, year, title in books:
    library.setdefault(genre, {}).setdefault(year, []).append(title)
```

```
# Add "StarQuest" to Sci-Fi 2026 directly
library["Sci-Fi"].setdefault(2026, []).append("StarQuest")

# Print every year's titles for "Fantasy"
for year in library["Fantasy"]:
    print(year, library["Fantasy"][year])
```

Open the editor, run this code, and submit—see how the structure and output look!

Python

```
library = {}
books = [
    ("Fantasy", 2025, "Dragonfall"),
    ("Sci-Fi", 2025, "Galactic Run"),
    ("Fantasy", 2026, "Nightrealm"),
    ("Sci-Fi", 2026, "Moonwatch")
]

for genre, year, title in books:
    library.setdefault(genre, {}).setdefault(year, []).append(title)

library["Sci-Fi"].setdefault(2026, []).append("StarQuest")
for libs in library:
    for year in library["Fantasy"]:
        print(year, library["Fantasy"][year])
```

Nice job with the setdefault structure—your data shape is correct!

Just a quick catch: your print loop goes over every genre, but then always prints all the "Fantasy" titles for each genre, so you're seeing the Fantasy list twice.

You only need this to print every year's titles for "Fantasy":

Python

```
for year in library["Fantasy"]:
    print(year, library["Fantasy"][year])
```

And if you want to double-check, print the whole structure:

Python

```
print(library)
```

Want to try this tiny adjustment and check that "StarQuest" is under Sci-Fi 2026?

no im about sik of it ad clelless

That's totally okay, Ron. This is a really tough topic, and feeling stuck happens to everyone. You made big progress and got the hardest part to work—even if it feels fuzzy right now.

Before we finish, here's what you just accomplished:

You built and updated nested data structures using real code!
You learned when to reach for lists, dicts, and combos like dict-of-lists or dict-of-dict-of-lists.
You practiced safe updating with setdefault to avoid errors with missing keys.
Want a super simple summary or a quick "cheat sheet" before we wrap up? Or would you rather just end here for today?

i dont even get the workdig of the ta at ha whole thigs a ess maybe next time

Thanks for sharing honestly, Ron. Sometimes brains just hit "full"—that's normal.

This is tough material; you're not alone in feeling that way.

If you want, next time we'll break it into simpler steps with way more visuals and real-life comparisons.

We can start wherever you want—no pressure to rush.

Just to confirm: do you want to stop and finish the session for today?

Python

```
library = {}
books = [
    ("Fantasy", 2025, "Dragonfall"),
    ("Sci-Fi", 2025, "Galactic Run"),
    ("Fantasy", 2026, "Nightrealm"),
    ("Sci-Fi", 2026, "Moonwatch")
]

for genre, year, title in books:
    library.setdefault(genre, {}).setdefault(year, []).append(title)

library["Sci-Fi"].setdefault(2026, []).append("StarQuest")
for year in library["Fantasy"]:
    print(year, library["Fantasy"][year])
```

You did it, Ron—you finished the session, even if it felt messy. That counts.

Today you:

Built, updated, and looped through tricky nested data structures.
Practiced using .keys(), .values(), .items(), and setdefault in more complex ways than most people see.
Saw how to use a dict of dicts for two-step lookup—even if it’s still a little foggy!
If anything sticks later, or you want to start again with a slower pace or different approach, I’m always here for another round. Rest up—you’ve earned it!

no you id it

Thanks for hanging in there, Ron. Remember, wrestling with complicated things is how real growth happens—even when it feels like a mess.

You put in the effort—that’s what matters.

We can always come back and find a way that works better for y

```
colors = {"primary": ["red", "blue"], "neutral": ["white"]}
for color_key in colors.keys():
    print(f'Key = {color_key}')
for color_values in colors.values():
    print(f'Value = {color_values}')
for color_key, color_value in colors.items():
       print(f'Key = {color_key} - Value = {color_value}')
colors.setdefault("neutral", []).append("black")
print(colors)

students = [
    {"id": 1, "name": "Ana", "grade": "B"},
    {"id": 2, "name": "Luis", "grade": "C"},
    {"id": 3, "name": "Mina", "grade": "A"}
]

for student in students:
    print(student["name"])
for student in students:
    if student["id"] == 2:
        student["grade"] = "A"
new_student = {"id": 4, "name": "Bob", "grade": "A"}
students.append(new_student)
print(students)
```

```
by_country = {}

people = [("Ana", "Mexico"), ("John", "USA"), ("Mina", "Japan"), ("Luis", "Mexico")]

for names, country in people:
    if country not in by_country:
        by_country[country] = []
    by_country[country].append(names)
by_country["USA"].append("Ron")
print(by_country)

#CLEANER VERSION
by_country = {}
people = [("Ana", "Mexico"), ("John", "USA"), ("Mina", "Japan"), ("Luis", "Mexico")]

for names, country in people:
    by_country.setdefault(country, []).append(names)
by_country.setdefault("USA", []).append("Ron")

print(by_country)

library = {}
books = [
    ("Fantasy", 2025, "Dragonfall"),
    ("Sci-Fi", 2025, "Galactic Run"),
    ("Fantasy", 2026, "Nightrealm"),
    ("Sci-Fi", 2026, "Moonwatch")
]

for genre, year, title in books:
    library.setdefault(genre, {}).setdefault(year, []).append(title)

library["Sci-Fi"].setdefault(2026, []).append("StarQuest")
for year in library["Fantasy"]:
    print(year, library["Fantasy"][year])
```